Author: PyShaala
Date: August 17, 2025
YouTube Channel: PyShaala

What are Lists?

In Python, a **list** is a versatile, ordered, and mutable collection of items. Lists are one of the most fundamental and widely used data types in Python.

Key Characteristics:

- **Ordered:** Items in a list have a defined order, and this order will not change unless explicitly modified. You can access items by their index.
- **Mutable:** Lists can be changed after they are created. You can add, remove, or modify elements within a list.
- **Allows Duplicates:** Lists can contain multiple items with the same value.
- **Heterogeneous:** Lists can hold items of different data types simultaneously (e.g., integers, strings, objects, and even other lists).

Common Uses:

Lists are used for a wide variety of tasks, such as:

- Storing collections of data (e.g., a list of student names, a list of temperatures).
- Implementing stacks and queues.
- Iterating over a sequence of items.
- Manipulating ordered data.

📝 Syntax of Creating Lists

Lists in Python are typically created using square brackets `[]`. You can create empty lists, lists with various data types, or even lists from other iterable objects.

Creating Lists with Square Brackets `[]`

This is the most common and straightforward way to create a list.

In [1]:
```
# 1. Empty list using square brackets
empty_list_sq = []
print(f"Empty list with []: {empty_list_sq}")
print(f"Type of empty_list_sq: {type(empty_list_sq)}")
```

```
Empty list with []: []
Type of empty_list_sq: <class 'list'>
```

In [2]:
```
# 2. List of integers
numbers = [1, 2, 3, 4, 5]
print(numbers)
```

```
[1, 2, 3, 4, 5]
```

In [3]:
```
# 3. List of strings
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

```
['apple', 'banana', 'cherry']
```

In [4]:
```
# 4. List with mixed data types
mixed_list = [1, "hello", 3.14, True]
```

```
print(mixed_list)
```

```
[1, 'hello', 3.14, True]
```

Note: Nested Lists

- A **nested list** is a list that contains other lists as its elements.
- They are often used to represent matrices or hierarchical data.

```
nested_list_example = [[1, 2], [3, 4], [5, 6]]
```

In [5]:
```
# 5. Nested list (a list containing other lists)
nested_list = [[1, 2], [3, 4], ["a", "b"]]

print(nested_list)
```

```
[[1, 2], [3, 4], ['a', 'b']]
```

Creating Lists using the `list()` Constructor

You can also create a list using the built-in `list()` constructor. This is particularly useful for creating an empty list or converting other iterable objects (like tuples, strings, sets, or ranges) into lists.

In [6]:
```
# 1. Using the list() constructor
list_constructor = list()
print(f"Empty list with list(): {list_constructor}")
print(f"Type of list_constructor: {type(list_constructor)}")
```

```
Empty list with list(): []
Type of list_constructor: <class 'list'>
```

In [7]:
```
# 2. Creating a list from a tuple
sample_tuple = (100, 200, 300)
list_from_iterable_tuple = list(sample_tuple)
print(f"List from tuple {sample_tuple}: {list_from_iterable_tuple}")
print(f"Type of list_from_iterable_tuple: {type(list_from_iterable_tuple)}")
```

```
List from tuple (100, 200, 300): [100, 200, 300]
Type of list_from_iterable_tuple: <class 'list'>
```

In [8]:
```
# 3. Creating a list from a string
sample_string = "pyshaala"
list_from_iterable_string = list(sample_string)
print(f"List from string '{sample_string}': {list_from_iterable_string}")
print(f"Type of list_from_iterable_string: {type(list_from_iterable_string)}")
```

```
List from string 'pyshaala': ['p', 'y', 's', 'h', 'a', 'a', 'l', 'a']
Type of list_from_iterable_string: <class 'list'>
```

In [9]:
```
# 4. Creating a list from a range
sample_range = range(7, 10) # Creates numbers 7, 8, 9
list_from_iterable_range = list(sample_range)
print(f"List from range {list(sample_range)}: {list_from_iterable_range}")
print(f"Type of list_from_iterable_range: {type(list_from_iterable_range)}")
```

```
List from range [7, 8, 9]: [7, 8, 9]
Type of list_from_iterable_range: <class 'list'>
```

Creating Lists using dynamic input

You can create a dynamic list using the `input() and eval() function`

In [10]:
```
my_list = eval(input("Enter a List")) # Enter a list from user input like [1,2,3]
print(my_list)
print(type(my_list))
```

```
[1, 2, 3]
<class 'list'>
```

Creating Lists using split() function

You can create a list using the `split() function`

In [11]:
```
my_string = "Learing python is very easy"
my_list = my_string.split()
print(my_list)
print(type(my_list))
```

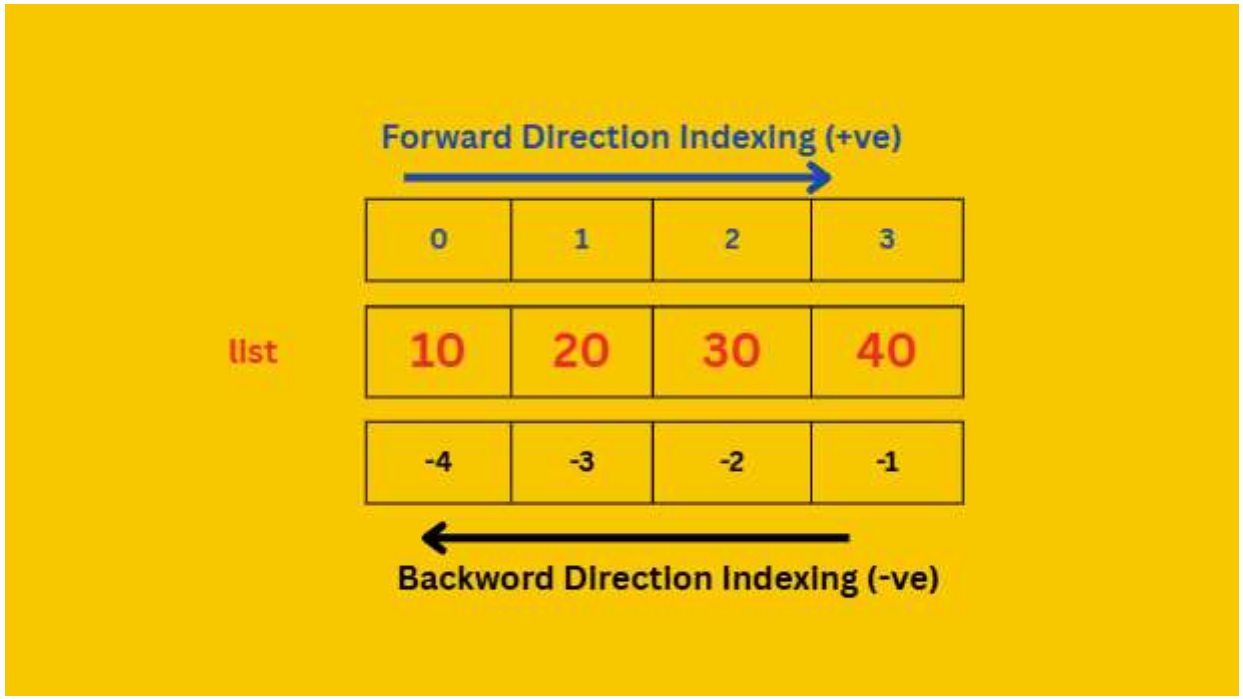
```
['Learing', 'python', 'is', 'very', 'easy']
<class 'list'>
```

🌎 Accessing elements of list

We can access elements of the list either by using `index` or by using `slice operator [start:stop:step]`

List Indexing

- List follow zero based index i.e index of first element is zro
- list supports both +ve and -ve indexes.
- +ve index means from Left to Right direction
- -ve index measn from Right to Left direction



In [12]:
```
my_list = [10, 20, 30, 40]
# print(my_list[2])  #10
# print(my_list[-2]) #40
# print(my_list[-5]) # IndexError: list index out of range
```

In [13]:
```
# Accesing Nested List Element
"""
[
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]
"""
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(f"Nested list (matrix): {matrix}")
print(f"First row: {matrix[0]}")
print(f"Element at [2][2]: {matrix[2][2]}") # Accessing element 9
```

```
Nested list (matrix): [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
First row: [1, 2, 3]
Element at [2][2]: 9
```

List Slicing

Slicing allows you to access a portion (a "slice") of a list. It creates a *new* list containing the requested elements. The syntax is `list[start:stop:step]`.

```
my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Basic slice
slice1 = my_list[2:6] # Elements from index 2 up to (but not including) 6: [2, 3, 4, 5]

# Slice from the beginning
slice2 = my_list[:5] # Elements from start up to (but not including) 5: [0, 1, 2, 3, 4]

# Slice to the end
slice3 = my_list[7:] # Elements from index 7 to the end: [7, 8, 9]

# Slice with a step
slice4 = my_list[::2] # Every second element: [0, 2, 4, 6, 8]
slice5 = my_list[1:8:3] # Elements from index 1 up to 8, stepping by 3: [1, 4, 7]

# Reverse a list using slicing
reversed_list = my_list[::-1] # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

In [14]:
```
numbers_for_slicing = list(range(15))
print(f"List for slicing: {numbers_for_slicing}")
print(f"Slice [5:10]: {numbers_for_slicing[5:10]}")
print(f"Slice [::3]: {numbers_for_slicing[::3]}")
print(f"Slice [1::2]: {numbers_for_slicing[1::2]}")
print(f"Reverse slice [::-1]: {numbers_for_slicing[::-1]}")
```

```
List for slicing: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
Slice [5:10]: [5, 6, 7, 8, 9]
Slice [::3]: [0, 3, 6, 9, 12]
Slice [1::2]: [1, 3, 5, 7, 9, 11, 13]
Reverse slice [::-1]: [14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

🌍 Real-Life Examples

- Lists are incredibly useful in everyday programming tasks and are used extensively to manage collections of data.
- In the below example we will demonstrating how lists can be used to model and manipulate data in real-world scenarios.

In [15]:
```
# Example : Managing a Shopping List 🛒

# Initialize a shopping list (list of items)
shopping_list = ["Apples", "Bananas", "Milk", "Bread"]
print(f"Initial shopping list: {shopping_list}")

# Add a new item to the list
shopping_list.append("Cheese")
print(f"After adding Cheese: {shopping_list}")

# Remove an item from the list
try:
    shopping_list.remove("Bananas")
    print(f"After removing Bananas: {shopping_list}")
except ValueError:
    print("Bananas not found in the list.")

# Access an item by its index
print(f"First item: {shopping_list[0]}")
print(f"Last item: {shopping_list[-1]}")

# Check if an item is in the list
```

```python
item_to_check = "Bananas"
if item_to_check in shopping_list:
    print(f"{item_to_check} is in the list.")
else:
    print(f"{item_to_check} is not in the list.")

# Get the number of items in the list
print(f"Number of items: {len(shopping_list)}")

# Iterate through the list
print("Items in the list:")
for item in shopping_list:
    print(f"- {item}")
```

```
Initial shopping list: ['Apples', 'Bananas', 'Milk', 'Bread']
After adding Cheese: ['Apples', 'Bananas', 'Milk', 'Bread', 'Cheese']
After removing Bananas: ['Apples', 'Milk', 'Bread', 'Cheese']
First item: Apples
Last item: Cheese
Bananas is not in the list.
Number of items: 4
Items in the list:
- Apples
- Milk
- Bread
- Cheese
```

⚙️ List Methods (Functions)

Python lists have a variety of built-in methods that allow you to efficiently modify, query, and manipulate list elements. Most list methods modify the list *in-place* and return `None`, while a few (like `copy()`) return a new list.

Here are some of the most commonly used and useful list methods and related operations:

- `.append(item)`: Adds an item to the end of the list.
- `.extend(iterable)`: Extends the list by appending all the items from an iterable (like another list, tuple, or string).
- `.insert(index, item)`: Inserts an item at a specified index.
- `.remove(item)`: Removes the first occurrence of an item from the list. Raises a `ValueError` if the item is not found.
- `.pop(index)`: Removes and returns the item at a specified index. If no index is given, it removes and returns the last item.

- `del list[index]` or `del list[slice]` : Removes items at a specific index or slice. `del` is a statement, not a method.
- `.clear()` : Removes all items from the list.
- `.index(item)` : Returns the index of the first occurrence of an item. Raises a `ValueError` if the item is not found.
- `.count(item)` : Returns the number of occurrences of an item in the list.
- `.sort()` : Sorts the list in ascending order in-place.
- `.reverse()` : Reverses the order of the list in-place.
- `.copy()` : Returns a shallow copy of the list. (Note: `list()` can also create a shallow copy).

- `.append(item)` : Adds an item to the end of the list.

In [16]:
```
# Examples of common list methods and operations

my_list = [10, 20, 30, 40]
print(f"Original list: {my_list}")

# .append(item)
my_list.append(50)
print(f"After append(50): {my_list}")
```

```
Original list: [10, 20, 30, 40]
After append(50): [10, 20, 30, 40, 50]
```

- `.insert(index, item)` : Inserts an item at a specified index.

In [17]:
```
my_list = [10, 20, 30, 40]
print(f"Original list: {my_list}")

# .insert(index, item)
my_list.insert(1, 15) # Insert 15 at index 1
print(f"After insert(1, 15): {my_list}")
```

```
Original list: [10, 20, 30, 40]
After insert(1, 15): [10, 15, 20, 30, 40]
```

Note:

- If the specified index is greater than max index then element will be inserted at last position.

- If the specified index is smaller than min index then element will be inserted at first position.

In [18]:
```
my_list = [10, 20, 30, 40, 20]
print(f"Original list: {my_list}")

# my_list.insert(50, 777) # Insert 15 at index 50
# print(f"After insert(50, 777): {my_list}")

my_list.insert(-10, 111) # Insert -1 at index -111
print(f"After insert(-10, 111): {my_list}")
```

```
Original list: [10, 20, 30, 40, 20]
After insert(-10, 111): [111, 10, 20, 30, 40, 20]
```

Differences between append() and insert()

append()	insert()
In List when we add any element it will come in last i.e. it will be last element.	In List we can insert any element in particular index number

- `.extend(iterable)` : Extends the list by appending all the items from an iterable (like another list, tuple, or string).

```
l1.extend(l2)
 #all items present in l2 will be added to l1
```

In [19]:
```
# .extend(iterable)
my_list = [10, 20, 30, 40, 20]
print(f"Original list: {my_list}")

another_list = [60, 70]
my_list.extend(another_list)
print(f"After extend({another_list}): {my_list}")
my_list
```

```
Original list: [10, 20, 30, 40, 20]
After extend([60, 70]): [10, 20, 30, 40, 20, 60, 70]
```

Out[19]: [10, 20, 30, 40, 20, 60, 70]

In [20]:
```python
my_list = [10, 20, 30, 40, 20]
print(f"Original list: {my_list}")
my_tuple = (111, 222, 'python')
my_list.extend(my_tuple)
print(f"After extending ({my_tuple}): {my_list}")
```

```
Original list: [10, 20, 30, 40, 20]
After extending ((111, 222, 'python')): [10, 20, 30, 40, 20, 111, 222, 'python']
```

In [21]:
```python
my_list = [10, 20, 30, 40, 20]
print(f"Original list: {my_list}")
my_string = 'python'
my_list.extend(my_string)
print(f"After extending ({my_string}): {my_list}")
```

```
Original list: [10, 20, 30, 40, 20]
After extending (python): [10, 20, 30, 40, 20, 'p', 'y', 't', 'h', 'o', 'n']

After extending (python): [10, 20, 30, 40, 20, 'p', 'y', 't', 'h', 'o', 'n']
```

- `.remove(item)` : Removes the first occurrence of an item from the list. Raises a `ValueError` if the item is not found.

In [22]:
```python
my_list = [10, 20, 30, 40, 20]
print(f"Original list: {my_list}")
# .remove(item)
try:
    my_list.remove(50) # Removes the first 20
    print(f"After remove(20): {my_list}")
except ValueError:
    print("Item not found for remove()")

# my_list.remove(50)
```

```
Original list: [10, 20, 30, 40, 20]
Item not found for remove()
```

`Note:` Before using remove() method first we have to check specified element present in the list or not by using `membership in operator`.

In [23]:
```
my_list = [10, 20, 30, 40, 20]
print(f"Original list: {my_list}")
element = 40
if element in my_list:
    my_list.remove(element)
    print(f"After remove({element}): {my_list}")
else:
    print(f"Element:{element} not found in list:{my_list}")
```

```
Original list: [10, 20, 30, 40, 20]
After remove(40): [10, 20, 30, 20]
```

- `.pop(index)` : Removes and returns the item at a specified index. If no index is given, it removes and returns the last item.

This is only function which manipulates list and returns some element.

In [24]:
```
# .pop(index)
my_list = [10, 20, 30, 40, 20]
popped_item = my_list.pop(0) # Remove and return item at index 0
print(f"After pop(0): {my_list}, Popped item: {popped_item}")
```

```
After pop(0): [20, 30, 40, 20], Popped item: 10
```

In [25]:
```
last_item = my_list.pop() # Remove and return the last item
print(f"After pop(): {my_list}, Last item: {last_item}")
```

```
After pop(): [20, 30, 40], Last item: 20
```

In [26]:
```
# If the list is empty then pop() function raises IndexError
empty_list = []
# print(empty_list.pop()) # IndexError: pop from empty list
```

Differences between remove() and pop()

remove()	pop()
1) We can use to remove special element from the List.	1) We can use to remove last element from the List.
2) It can't return any value.	2) It returned removed element.
3) If special element not available then we get VALUE ERROR.	3) If List is empty then we get Error.

- `del list[index]` or `del list[slice]` : Removes items at a specific index or slice. `del` is a statement, not a method.

In [27]:
```
# del statement
del my_list[1] # Delete item at index 1
print(f"After del my_list[1]: {my_list}")
```

```
After del my_list[1]: [20, 40]
```

In [28]:
```
my_list = [10, 20, 30, 40, 20, 50, 60, 70]
del my_list[1:5] # Delete items from index 1 up to (but not including) 3
print(f"{my_list}")
```

```
[10, 50, 60, 70]
```

- `.clear()` : Removes all items from the list.

In [29]:
```
# .clear()
my_list.clear()
print(f"After clear(): {my_list}")
# Re-initialize list for further examples
my_list = [10, 20, 30, 40, 20, 50, 60, 70]
print(f"List after re-initialization: {my_list}")
```

```
After clear(): []
List after re-initialization: [10, 20, 30, 40, 20, 50, 60, 70]
```

- `.index(item)` : Returns the index of the first occurrence of an item. Raises a `ValueError` if the item is not found.

In [31]:
```
# .index(item)
# try:
    # index_of_30 = my_list.index(30)
    # print(f"Index of 30: {index_of_30}")
    # index_of_20 = my_list.index(20) # Returns the first occurrence
    # print(f"Index of first 20: {index_of_20}")
#     index_of_99 = my_list.index(99) # This would raise ValueError
# except ValueError as e:
#     print(f"Error using index(): {e}")
# index_of_99 = my_list.index(99)
```

- `.count(item)` : Returns the number of occurrences of an item in the list.

In [32]:
```
# .count(item)
print(my_list)
count_of_20 = my_list.count(20)
print(f"Count of 20: {count_of_20}")
count_of_99 = my_list.count(99)
print(f"Count of 99: {count_of_99}")
```

```
[10, 20, 30, 40, 20, 50, 60, 70]
Count of 20: 2
Count of 99: 0
```

- `.sort()` : **Sorts the list in ascending order in-place.**
- In list by default insertion order is preserved.
- If want to sort the elements of list according to default natural sorting order then we should go for `sort()` method.
- `For numbers` --> default natural sorting order is Ascending Order
- `For Strings` --> default natural sorting order is Alphabetical Order

In [33]:
```
# .sort()
unsorted_list = [5, 2, 8, 1.0, 9, 4]
print(f"Unsorted list: {unsorted_list}")
unsorted_list.sort() # Sorts in-place
print(f"After sort(): {unsorted_list}")
```

```
Unsorted list: [5, 2, 8, 1.0, 9, 4]
After sort(): [1.0, 2, 4, 5, 8, 9]
```

In [34]:
```
str_list = ["Dog","Banana","Cat","Apple", 'apple']
str_list.sort()
print(str_list) #['Apple','Banana','Cat','Dog']
```

```
['Apple', 'Banana', 'Cat', 'Dog', 'apple']
```

In [35]:
```
# .sort(reverse=True) it will sort numbers in descending order
unsorted_list = [5, 2, 8, 1, 9, 4]
print(f"Unsorted list: {unsorted_list}")
unsorted_list.sort(reverse=True)
print(f"After sort(reverse=True): {unsorted_list}")
```

```
Unsorted list: [5, 2, 8, 1, 9, 4]
After sort(reverse=True): [9, 8, 5, 4, 2, 1]
```

- `Note:` To use sort() function, compulsory list should contain only homogeneous elements. `otherwise we will get TypeError`

In [36]:
```
mix_list = [20,10,"A","B"]
# mix_list.sort() #TypeError: '<' not supported between instances of 'str' and 'int'
```

```
print(mix_list)
```

```
[20, 10, 'A', 'B']
```

- `.reverse()` : Reverses the order of the list in-place.

In [37]:
```
# .reverse()
unsorted_list = [5, 2, 8, 1, 9, 4]
print(f"Unsorted list: {unsorted_list}")
unsorted_list.reverse() # Reverses in-place
print(f"After reverse(): {unsorted_list}")
```

```
Unsorted list: [5, 2, 8, 1, 9, 4]
After reverse(): [4, 9, 1, 8, 2, 5]
```

- `.copy()` : Returns a shallow copy of the list.

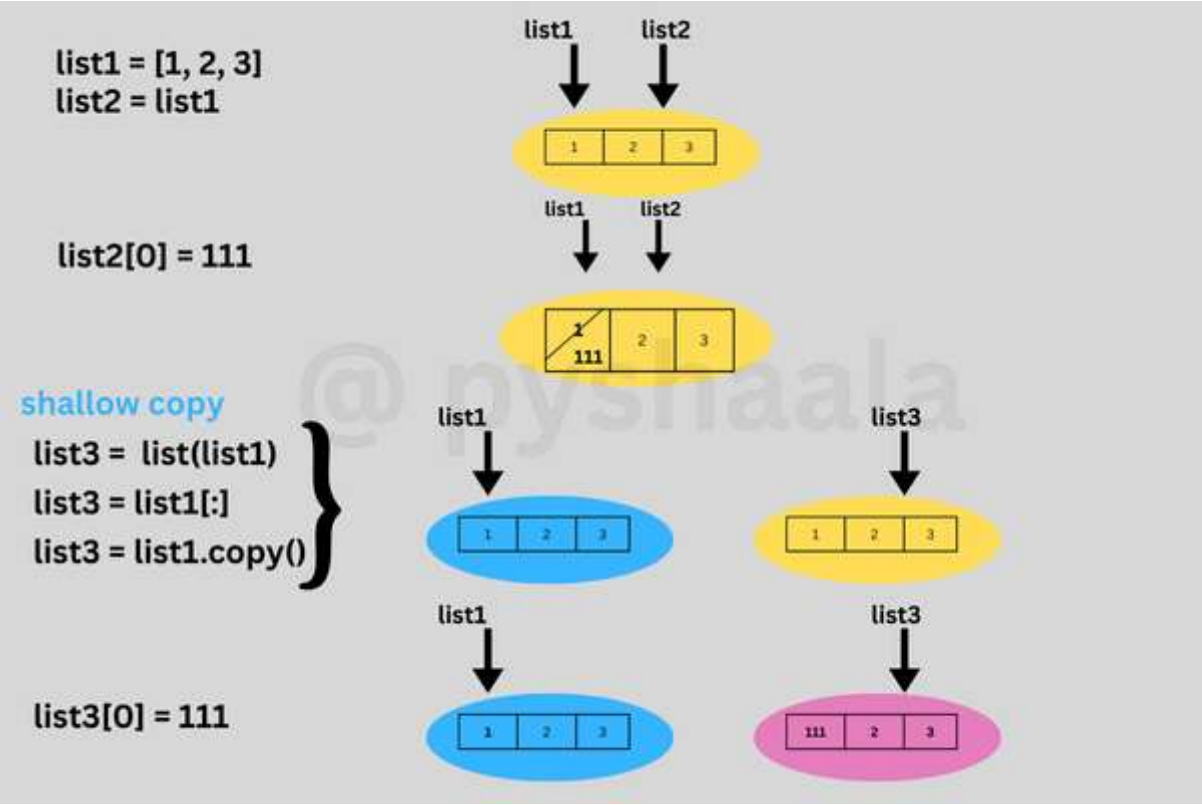
In [38]:
```
list1 = [1, 2, 3]
print(f"Original list1:{list1} ")
list2 = list1
print(f"list1 coping to list2:{list2}")
```

```
Original list1:[1, 2, 3]
list1 coping to list2:[1, 2, 3]
```

In [39]:
```
print(id(list1))
print(id(list2))

print("modifying list2 lets check both list now")
list1[1] = 555
print(f"After modifying list2 \n list1:{list1} \n list2:{list2}")
```

```
2169659151296
2169659151296
modifying list2 lets check both list now
After modifying list2
 list1:[1, 555, 3]
 list2:[1, 555, 3]
```



3 ways to create a list copy ---> but affect if nested list we modify

1. list()
2. slicing[:]
3. .cpoy()

In [40]:
```
# 1. list()
list1 = [1, 2, 3, [55, 66, 77]]
print(f"Original list1:{list1} ")
list3 = list(list1)
print(f"list3:{list3}")
print(id(list1)==id(list3))
print('-'*30)

print("modifying list3")
```

```
list3[3].append(963)
print(f"After modifying list3: \n list1:{list1} \n list3:{list3}")
print(f"Address: \n list1:{id(list1)} \n list3:{id(list3)}")
```

```
Original list1:[1, 2, 3, [55, 66, 77]]
list3:[1, 2, 3, [55, 66, 77]]
False
------------------------------
modifying list3
After modifying list3:
 list1:[1, 2, 3, [55, 66, 77, 963]]
 list3:[1, 2, 3, [55, 66, 77, 963]]
Address:
 list1:2169652749952
 list3:2169659043264
```

In [41]:
```
# 2. slicing[:]
list1 = [1, 2, 3, ['a', 'b']]
print(f"Original list1:{list1} ")
list4 = list1[:]
print(f"list4:{list4}")
print("modifying list4")
list4[3].append('New item')
print(f"After modifying list4: \n list1:{list1} \n list4:{list4}")
```

```
Original list1:[1, 2, 3, ['a', 'b']]
list4:[1, 2, 3, ['a', 'b']]
modifying list4
After modifying list4:
 list1:[1, 2, 3, ['a', 'b', 'New item']]
 list4:[1, 2, 3, ['a', 'b', 'New item']]
```

In [42]:
```
# 3.  .copy() (Shallow Copy)
original_list = [1, 2, [3, 4]]

shallow_copy = original_list.copy() # Creates a shallow copy

print(f"Original list: {original_list} id: {id(original_list)}")
print(f"Shallow copy: {shallow_copy} id: {id(shallow_copy)}")

# Modify the shallow copy - affects mutable nested objects
# shallow_copy[0] = 99 # Modifies the copy, not the original
shallow_copy[2].append(5) # Modifies the nested list in *both* original and copy
```

```
print(f"Original list after modifying copy: {original_list}") # Nested list is affected
print(f"Shallow copy after modification: {shallow_copy}")

# Using list() for shallow copy
another_shallow_copy = list(original_list)
print(f"Another shallow copy using list(): {another_shallow_copy}")
```

```
Original list: [1, 2, [3, 4]] id: 2169652787392
Shallow copy: [1, 2, [3, 4]] id: 2169658742784
Original list after modifying copy: [1, 2, [3, 4, 5]]
Shallow copy after modification: [1, 2, [3, 4, 5]]
Another shallow copy using list(): [1, 2, [3, 4, 5]]
```

Shallow Copy vs. Deep Copy

Understanding how copies of lists work is crucial because lists are mutable.

- **Assignment (`=`):** Simple assignment does *not* create a copy. It creates a new variable that refers to the *same* list object in memory. Changes to one variable affect the other.
- **Shallow Copy (`.copy()` or `list()`):** Creates a *new* list object. However, if the original list contains mutable objects (like other lists), the copy will contain references to the *same* nested mutable objects. Changes to nested mutable objects in the copy *will* affect the original, and vice-versa.
- **Deep Copy (`copy.deepcopy()`):** Creates a *new* list object and recursively creates copies of all nested mutable objects. Changes to the deep copy (including nested objects) do *not* affect the original list. You need to import the `copy` module for this.

```
import copy

original_list = [1, 2, [3, 4]]

# Assignment - not a copy
assigned_list = original_list
assigned_list[0] = 99
print(f"Original after assignment change: {original_list}") # Output: [99, 2, [3, 4]] - Original is
affected

# Shallow Copy
```

```
shallow_copy = original_list.copy() # Or list(original_list)
shallow_copy[0] = 100 # Changes shallow_copy, not original (for immutable element)
shallow_copy[2].append(5) # Changes the nested list in *both* original and shallow_copy

print(f"Original after shallow copy nested change: {original_list}") # Output: [99, 2, [3, 4, 5]] -
Nested list affected
print(f"Shallow copy after nested change: {shallow_copy}")         # Output: [100, 2, [3, 4, 5]] - Nested
list affected

# Deep Copy
original_list = [1, 2, [3, 4]] # Reset original
deep_copy = copy.deepcopy(original_list)
deep_copy[0] = 100 # Changes deep_copy, not original
deep_copy[2].append(5) # Changes the nested list ONLY in deep_copy

print(f"Original after deep copy nested change: {original_list}") # Output: [1, 2, [3, 4]] - Original NOT
affected
print(f"Deep copy after nested change: {deep_copy}")         # Output: [100, 2, [3, 4, 5]]
```

In [43]:
```
# For deep copy, you need the 'copy' module
import copy

original_list = [1, 2, [3, 4]]
print(f"Original list1:{original_list} ")
deep_copy = copy.deepcopy(original_list)
print(f"Deep copy: {deep_copy}")
deep_copy[2].append(6) # Modifies only the deep copy's nested list
print(f"Original list after modifying deep copy: {original_list}") # Original is NOT affected
print(f"Deep copy after modification: {deep_copy}")
```

```
Original list1:[1, 2, [3, 4]]
Deep copy: [1, 2, [3, 4]]
Original list after modifying deep copy: [1, 2, [3, 4]]
Deep copy after modification: [1, 2, [3, 4, 6]]
```

In [44]:
```
# Shallow vs Deep Copy demonstration
import copy

list_with_nested = [1, [2, 3]]
shallow = list_with_nested.copy()
deep = copy.deepcopy(list_with_nested)
```

```
print(f"Original: {list_with_nested}")
print(f"Shallow: {shallow}")
print(f"Deep: {deep}")

# Modify nested list in shallow copy
shallow[1].append(4)
print(f"Original after shallow nested append: {list_with_nested}")
print(f"Shallow after nested append: {shallow}")
print(f"Deep after shallow nested append: {deep}") # Deep copy is unaffected

# Modify nested list in deep copy
deep[1].append(5)
print(f"Original after deep nested append: {list_with_nested}") # Original is unaffected
print(f"Shallow after deep nested append: {shallow}")           # Shallow is unaffected
print(f"Deep after nested append: {deep}")
```

```
Original: [1, [2, 3]]
Shallow: [1, [2, 3]]
Deep: [1, [2, 3]]
Original after shallow nested append: [1, [2, 3, 4]]
Shallow after nested append: [1, [2, 3, 4]]
Deep after shallow nested append: [1, [2, 3]]
Original after deep nested append: [1, [2, 3, 4]]
Shallow after deep nested append: [1, [2, 3, 4]]
Deep after nested append: [1, [2, 3, 5]]
```

🧩 List Operations (Concatenation, Repetition, Membership)

In [45]:
```
list1 = [1, 2, 3]
list2 = [4, 5, 6, 'abc']
print(list1 + list2)
```

```
[1, 2, 3, 4, 5, 6, 'abc']
```

In [46]:
```
list2 * 2
```

Out[46]:
```
[4, 5, 6, 'abc', 4, 5, 6, 'abc']
```

In [47]:
```
print(list1)
print(list2)
```

```
print('abc' in list1)
print('abc' in list2)
print('abc' not in list2)
```

```
[1, 2, 3]
[4, 5, 6, 'abc']
False
True
False
```

🧩 More List Concepts

Beyond basic methods, Python lists offer powerful features like comprehensions, unpacking, and flexible access methods.

List Comprehensions

List comprehensions provide a concise way to create lists.

- They consist of brackets containing an expression followed by a `for` clause, and then zero or more `for` or `if` clauses.
- They are often more readable and faster than traditional loops for creating lists.

Syntax `for` in List Comprehension:

new_list = [expression for item in iterable]

In [48]:
```
numbers = range(1,11)
print(numbers)
print(type(numbers))
my_list = []
for number in numbers:
    my_list.append(number)

print(my_list)
```

```
range(1, 11)
<class 'range'>
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [49]:
```
my_list2 = [number for number in range(1,11)]
print(my_list2)

squares = [x**2 for x in my_list2]
print(squares)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Syntax `for if` in List Comprehension:

- Contrast with if (filtering) in List Comprehension:
- When only an if condition is used (without an else), it acts as a filter, placed after the for loop, to include only elements that satisfy the condition.

`filtered_list = [expression for item in iterable if condition]`

- Example: To create a list containing only even numbers:

In [50]:
```
numbers = [1, 2, 3, 4, 5]
even_numbers = [num for num in numbers if num % 2 == 0]
print(even_numbers)
```

```
[2, 4]
```

Syntax `for if-else` in List Comprehension:

new_list = [expression_if_true if condition else expression_if_false for item in iterable]

Explanation:

- expression_if_true: The value or expression to be included in new_list if condition is True.
- condition: The condition that is evaluated for each item in the iterable.
- expression_if_false: The value or expression to be included in new_list if condition is False.
- for item in iterable: The loop that iterates through each item in the iterable (e.g., a list, tuple, range).

Example:

To create a list where even numbers are replaced by "Even" and odd numbers by "Odd":

In [51]:
```
numbers = [1, 2, 3, 4, 5]
result = ["Even" if num % 2 == 0 else "Odd" for num in numbers]
print(result)
```

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Example:

In [52]:
```
# Nested list comprehension
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix)
flattened_list = [num for row in matrix for num in row] # Flattens a nested list
print(flattened_list)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [53]:
```
# List comprehension with if condition
fruits = ["apple", "banana", "cherry", "date"]
filtered_list = [item  for item in fruits if len(item) > 5]
print(f"Filtered list using comprehension: {filtered_list}")
```

```
Filtered list using comprehension: ['banana', 'cherry']
```

Unpacking Lists

- **Unpacking** allows you to assign elements of a list (or other iterable) to multiple variables in a single line.
- The number of variables must match the number of elements in the list.
- You can also use the `*` operator to unpack remaining elements into a list.

```
# Basic unpacking
a, b, c = [1, 2, 3]

# Unpacking with the * operator (Python 3+)
first, *middle, last = [1, 2, 3, 4, 5] # first is 1, middle is [2, 3, 4], last is 5
```

In [54]:
```
# Unpacking
coordinates = [10, 20]
```

```
x, y = coordinates
print(f"Unpacked coordinates: x={x}, y={y}")

data = [1, 2, 3, 4, 5, 6]
first, *rest, last = data
print(f"Unpacking with *: first={first}, rest={rest}, last={last}")
```

```
Unpacked coordinates: x=10, y=20
Unpacking with *: first=1, rest=[2, 3, 4, 5], last=6
```

`zip()` Function

- The `zip()` function takes multiple iterables (like lists) and aggregates elements from each into tuples.
- It returns an iterator of tuples.
- The iteration stops when the shortest input iterable is exhausted.

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
list3 = [True, False, True, False] # Longer list

zipped_list = list(zip(list1, list2, list3)) # Convert iterator to list
# Result: [(1, 'a', True), (2, 'b', False), (3, 'c', True)] - Stops at shortest list (list1/list2)
```

In [55]:
```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
list3 = [True, False, True, False] # Longer list

zipped_list = list(zip(list1, list2, list3)) # Convert iterator to list
# Result: [(1, 'a', True), (2, 'b', False), (3, 'c', True)] - Stops at shortest list (list1/list2)
zipped_list
```

Out[55]:
```
[(1, 'a', True), (2, 'b', False), (3, 'c', True)]
```

In [56]:
```
# zip()
names = ["Shreya", "Jubin", "Arjit"]
ages = [35, 29, 32]
zipped_data = list(zip(names, ages))
print(f"Zipped names and ages: {zipped_data}")
```

```
Zipped names and ages: [('Shreya', 35), ('Jubin', 29), ('Arjit', 32)]
```

In [57]:
```
dict1 = dict(zipped_data)
print(dict1)
```

```
{'Shreya': 35, 'Jubin': 29, 'Arjit': 32}
```

`enumerate()` Function

- The `enumerate()` function adds a counter to an iterable and returns it as an enumerate object.
- This is commonly used in loops to get both the index and the value of each item.

```
fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(f"Index {index}: {fruit}")
# Output:
# Index 0: apple
# Index 1: banana
# Index 2: cherry
```

In [58]:
```
fruits = ["apple", "banana", "cherry"]
for i in range(len(fruits)):
    print(f"Index {i}: {fruits[i]}")
```

```
Index 0: apple
Index 1: banana
Index 2: cherry
```

In [59]:
```
for index, fruit in enumerate(fruits,2):
    print(f"Index {index}: {fruit}")
```

```
Index 2: apple
Index 3: banana
Index 4: cherry
```

In [60]:
```
# enumerate()
colors = ["red", "green", "blue"]
print("Using enumerate():")
for index, color in enumerate(colors):
    print(f"{color[0]}: {color}")
```

```
Using enumerate():
r: red
g: green
b: blue
```

✅ Do's and ❌ Don'ts

Following these guidelines can help you write better, more reliable code when working with Python lists.

✅ Do's

- **Do** use lists when you need an ordered, mutable collection of items that may contain duplicates and mixed data types.
- **Do** use list comprehensions for creating new lists concisely and efficiently based on existing iterables.
- **Do** use `.append()` for adding single items to the end of a list.
- **Do** use `.extend()` for adding multiple items from another iterable to the end of a list.
- **Do** use `.insert(index, item)` when you need to add an item at a specific position other than the end.
- **Do** use `.remove(item)` to remove the first occurrence of a specific value.
- **Do** use `.pop()` to remove an item by its index and retrieve its value, especially for stack-like behavior.
- **Do** use `del` for removing items by index or slice, or for deleting the list itself.
- **Do** use `.copy()` or `list()` for shallow copies if you understand the implications for nested mutable objects.
- **Do** use `copy.deepcopy()` from the `copy` module when you need a truly independent copy, including nested mutable objects.
- **Do** use `in` keyword to check for membership (`item in my_list`).
- **Do** use `enumerate()` when you need both the index and value while iterating.
- **Do** use `zip()` to iterate over multiple lists simultaneously.
- **Do** use meaningful variable names for your lists.

❌ Don'ts

- **Don't** modify a list while iterating over it using a simple `for` loop on the original list, as this can lead to unexpected skipping of elements or errors. Iterate over a copy (`my_list[:]`) or build a new list instead.
- **Don't** use repeated concatenation (`+`) in a loop to add items to a list; use `.append()` or `.extend()` instead for better performance.

- **Don't** compare floating-point numbers within a list for exact equality (`==`); use a tolerance check.
- **Don't** confuse assignment (`=`) with copying; assignment creates a reference, not a new list.
- **Don't** forget that shallow copies still share references to nested mutable objects with the original list.
- **Don't** use `remove()` or `index()` on an empty list or for items that might not be present without error handling (e.g., `try-except`).
- **Don't** rely on the order of elements in a list if it's not explicitly maintained; lists are ordered, but operations like sorting change the order.

 Practice Exercises

Test your understanding of Python lists and their operations with the following exercises. Write your code in the cells below each problem description.

Exercise 1: Basic List Operations

1. Create a list called `fruits` containing the strings "apple", "banana", "cherry".
2. Add "orange" to the end of the `fruits` list.
3. Insert "grape" at the beginning of the `fruits` list.
4. Remove "banana" from the list.
5. Print the final `fruits` list.

Exercise 2: Modifying Lists by Index and Value

1. Create a list called `numbers` containing `[10, 20, 30, 40, 50, 20]` .
2. Remove the item at index 2 using `.pop()` . Print the removed item and the list.
3. Remove the last item using `.pop()` . Print the removed item and the list.
4. Remove the first occurrence of the value 20 using `.remove()` . Print the list.
5. Delete the item at index 0 using `del` . Print the list.

Exercise 3: List Information and Order

1. Create a list called `mixed_data` with elements `[5, "hello", 3.14, 5, True, "hello"]`.
2. Find the index of the first occurrence of "hello". Print the index.
3. Count how many times the value 5 appears in the list. Print the count.
4. Create a list of numbers `unsorted_numbers = [9, 1, 8, 2, 7, 3]`. Sort this list in-place and print it.
5. Reverse the sorted `unsorted_numbers` list in-place and print it.

Exercise 4: List Comprehensions and Slicing

1. Create a list of squares of numbers from 0 to 9 using a list comprehension. Print the list.
2. Create a new list from the squares list containing only the even squares using a list comprehension. Print the new list.
3. Take the squares list and create a slice containing elements from index 3 to 7 (inclusive). Print the slice.
4. Take the squares list and create a slice containing every third element starting from the beginning. Print the slice.

Exercise 5: Unpacking, zip(), and enumerate()

1. Create a list `point = [10, 20, 30]`. Unpack its elements into three variables `x`, `y`, and `z`. Print `x, y, z`.
2. Create two lists: `names = ["Alice", "Bob"]` and `ages = [25, 30]`. Use `zip()` to combine them and convert the result to a list of tuples. Print the zipped list.
3. Iterate over the `names` list using `enumerate()` and print each index and name in the format "Index: [index], Name: [name]".

Exercise 6: Shallow vs Deep Copy

1. Create a nested list `original = [1, [2, 3]]`.
2. Create a shallow copy of `original` using `.copy()`.
3. Create a deep copy of `original` using `copy.deepcopy()` (remember to import `copy`).
4. Append the number 4 to the nested list within the shallow copy (`shallow[1].append(4)`).
5. Append the number 5 to the nested list within the deep copy (`deep[1].append(5)`).
6. Print the `original`, `shallow`, and `deep` lists after these modifications. Observe how the changes affected each list.

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

✅ Solutions

Here are the solutions to the practice exercises. Try to solve them yourself before looking at the solutions!

Exercise 1 Solution: Basic List Operations

In [61]:
```
# Exercise 1 Solution: Basic List Operations

# 1. Create a list called fruits
fruits = ["apple", "banana", "cherry"]
print(f"Initial list: {fruits}")

# 2. Add "orange" to the end
fruits.append("orange")
print(f"After append('orange'): {fruits}")

# 3. Insert "grape" at the beginning
fruits.insert(0, "grape")
print(f"After insert(0, 'grape'): {fruits}")

# 4. Remove "banana"
try:
    fruits.remove("banana")
    print(f"After remove('banana'): {fruits}")
except ValueError:
    print("Banana not found in the list.")
```

```
# 5. Print the final list
print(f"Final fruits list: {fruits}")
```

```
Initial list: ['apple', 'banana', 'cherry']
After append('orange'): ['apple', 'banana', 'cherry', 'orange']
After insert(0, 'grape'): ['grape', 'apple', 'banana', 'cherry', 'orange']
After remove('banana'): ['grape', 'apple', 'cherry', 'orange']
Final fruits list: ['grape', 'apple', 'cherry', 'orange']
```

Exercise 2 Solution: Modifying Lists by Index and Value

In [67]:
```
# Exercise 2 Solution: Modifying Lists by Index and Value

# 1. Create a list called numbers
numbers = [10, 20, 30, 40, 50, 20]
print(f"Initial list: {numbers}")

# 2. Remove item at index 2 using .pop()
popped_item_at_index_2 = numbers.pop(2)
print(f"After pop(2): {numbers}, Popped item: {popped_item_at_index_2}")

# 3. Remove the last item using .pop()
last_popped_item = numbers.pop()
print(f"After pop(): {numbers}, Popped item: {last_popped_item}")

# 4. Remove the first occurrence of value 20 using .remove()
try:
    numbers.remove(20)
    print(f"After remove(20): {numbers}")
except ValueError:
    print("Value 20 not found for remove()")


# 5. Delete the item at index 0 using del
del numbers[0]
print(f"After del numbers[0]: {numbers}")
```

```
Initial list: [10, 20, 30, 40, 50, 20]
After pop(2): [10, 20, 40, 50, 20], Popped item: 30
After pop(): [10, 20, 40, 50], Popped item: 20
After remove(20): [10, 40, 50]
After del numbers[0]: [40, 50]
```

Exercise 3 Solution: List Information and Order

In [68]:
```python
# Exercise 3 Solution: List Information and Order

# 1. Create a list called mixed_data
mixed_data = [5, "hello", 3.14, 5, True, "hello"]
print(f"List: {mixed_data}")

# 2. Find the index of the first occurrence of "hello"
try:
    hello_index = mixed_data.index("hello")
    print(f"Index of first 'hello': {hello_index}")
except ValueError:
    print("'hello' not found in the list.")

# 3. Count how many times the value 5 appears
count_of_5 = mixed_data.count(5)
print(f"Count of 5: {count_of_5}")

# 4. Create and sort a list of numbers
unsorted_numbers = [9, 1, 8, 2, 7, 3]
print(f"Unsorted numbers: {unsorted_numbers}")
unsorted_numbers.sort() # Sorts in-place
print(f"After sort(): {unsorted_numbers}")

# 5. Reverse the sorted list
unsorted_numbers.reverse() # Reverses in-place
print(f"After reverse(): {unsorted_numbers}")
```

```
List: [5, 'hello', 3.14, 5, True, 'hello']
Index of first 'hello': 1
Count of 5: 2
Unsorted numbers: [9, 1, 8, 2, 7, 3]
After sort(): [1, 2, 3, 7, 8, 9]
After reverse(): [9, 8, 7, 3, 2, 1]
```

Exercise 4 Solution: List Comprehensions and Slicing

In [69]:
```
# Exercise 4 Solution: List Comprehensions and Slicing

# 1. Create a list of squares using list comprehension
squares = [x**2 for x in range(10)]
print(f"Squares from 0 to 9: {squares}")

# 2. Create a new list of even squares
even_squares = [s for s in squares if s % 2 == 0]
print(f"Even squares: {even_squares}")

# 3. Create a slice from index 3 to 7 (inclusive)
slice_3_to_7 = squares[3:8] # Remember slice stop index is exclusive
print(f"Slice from index 3 to 7: {slice_3_to_7}")

# 4. Create a slice with every third element
every_third = squares[::3]
print(f"Every third element: {every_third}")
```

```
Squares from 0 to 9: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
Even squares: [0, 4, 16, 36, 64]
Slice from index 3 to 7: [9, 16, 25, 36, 49]
Every third element: [0, 9, 36, 81]
```

Exercise 5 Solution: Unpacking, zip(), and enumerate()

In [70]:
```
# Exercise 5 Solution: Unpacking, zip(), and enumerate()

# 1. Unpack a list
point = [10, 20, 30]
x, y, z = point
print(f"Unpacked: x={x}, y={y}, z={z}")

# 2. Use zip()
names = ["Alice", "Bob"]
ages = [25, 30]
zipped_data = list(zip(names, ages))
print(f"Zipped names and ages: {zipped_data}")
```

```
# 3. Use enumerate()
colors = ["red", "green", "blue"]
print("Using enumerate():")
for index, color in enumerate(colors):
    print(f"Index: {index}, Name: {color}")
```

```
Unpacked: x=10, y=20, z=30
Zipped names and ages: [('Alice', 25), ('Bob', 30)]
Using enumerate():
Index: 0, Name: red
Index: 1, Name: green
Index: 2, Name: blue
```

Exercise 6 Solution: Shallow vs Deep Copy

In [71]:
```
# Exercise 6 Solution: Shallow vs Deep Copy

import copy

# 1. Create a nested list
original = [1, [2, 3]]
print(f"Original: {original}")

# 2. Create a shallow copy
shallow = original.copy()
print(f"Shallow copy: {shallow}")

# 3. Create a deep copy
deep = copy.deepcopy(original)
print(f"Deep copy: {deep}")

# 4. Append 4 to nested list in shallow copy
shallow[1].append(4)
print(f"After shallow[1].append(4):")
print(f"Original: {original}") # Affected
print(f"Shallow: {shallow}")   # Affected
print(f"Deep: {deep}")         # Not affected

# 5. Append 5 to nested list in deep copy
deep[1].append(5)
print(f"After deep[1].append(5):")
```

```
print(f"Original: {original}") # Not affected
print(f"Shallow: {shallow}")   # Affected (shares nested list with original)
print(f"Deep: {deep}")         # Affected
```

```
Original: [1, [2, 3]]
Shallow copy: [1, [2, 3]]
Deep copy: [1, [2, 3]]
After shallow[1].append(4):
Original: [1, [2, 3, 4]]
Shallow: [1, [2, 3, 4]]
Deep: [1, [2, 3]]
After deep[1].append(5):
Original: [1, [2, 3, 4]]
Shallow: [1, [2, 3, 4]]
Deep: [1, [2, 3, 5]]
```

✨ Summary of Covered Topics

- List Definition & Explanation
- Syntax of Creating list & Number of Ways Creating list
- Accessing List
- Real-life Examples
- List Methods (Functions)
- List Operations
- More List Concepts
- List comprehensions, unpacking, Zip(), enumerate()
- Do’s and Don’ts
- Practice Exercises
- Solutions
- Interview Questions

❓ Interview Questions

Here are some common interview questions related to Python lists:

1. What is a Python list? What are its key characteristics (ordered, mutable, etc.)?

2. How do you add an element to the end of a list? How do you insert an element at a specific index?
3. Explain the difference between `.remove()` and `.pop()` when removing elements from a list.
4. How do you remove an element by its index using the `del` statement?
5. How can you check if a specific item is present in a list?
6. How do you find the index of the first occurrence of an item in a list? What happens if the item is not found?
7. How do you count the number of times an item appears in a list?
8. Explain the difference between `.sort()` and `sorted()` for sorting lists.
9. What is list slicing? How do you get a sublist containing elements from index 2 up to (but not including) index 5?
10. How do you reverse a list using slicing?
11. What is a list comprehension? Provide a simple example.
12. Explain unpacking in the context of lists. How does the `*` operator work in unpacking?
13. What is a nested list? How do you access an element in a nested list?
14. Explain the purpose of the `zip()` function when working with lists.
15. Explain the purpose of the `enumerate()` function and how it's used in loops.
16. What is the difference between a shallow copy and a deep copy of a list? When would you use one over the other?
17. How do you create a shallow copy of a list? How do you create a deep copy?

🐥 Beginner Level Questions

These questions cover the basic syntax, concepts, and common operations of lists.

1. What is a list in Python? How is it different from a string?
2. How do you create an empty list?
3. How can you access elements from a list? Explain both positive and negative indexing.
4. What is list slicing? Give an example of how to get a sublist from a list.
5. Are lists mutable or immutable? Explain what this means.
6. Explain the purpose of the `append()` and `insert()` methods. What is the key difference between them?
7. How can you remove an element from a list? What is the difference between `remove()` and `pop()`?
8. How do you find the number of elements in a list?
9. How can you check if an element exists in a list?
10. What does the `clear()` method do?

💪 Intermediate Level Questions

These questions test the candidate's understanding of more advanced list methods, list comprehensions, and common programming patterns.

1. What is the difference between `append()` and `extend()`? When would you use one over the other?
2. How do you sort a list? Explain the difference between `list.sort()` and the `sorted()` built-in function.
3. Explain list comprehensions. Why are they considered a more "Pythonic" way to create lists? Provide an example.
4. How do you copy a list in Python? Explain the difference between a shallow copy and a deep copy.
5. Write a program to reverse a list without using the built-in `list.reverse()` method.
6. How can you use a list to implement a stack? What about a queue? What methods would you use for each?
7. How do you unpack a list into separate variables? Give an example.
8. What happens when you add two lists together using the `+` operator?
9. Explain how you would remove all duplicate elements from a list.
10. Can a list contain elements of different data types? Provide an example.

🧠 Advanced Level Questions

These questions focus on performance, memory management, and complex problem-solving with lists.

1. Discuss the time complexity of the following list operations: `append()`, `insert()` at the beginning, `pop()` from the end, and `pop()` from a specific index.
2. How is memory allocated for a Python list? Explain what happens when a list grows beyond its current capacity.
3. Write a function to merge two sorted lists into a single sorted list. Analyze the time complexity of your solution.
4. What is the difference between a list and a tuple? When would you choose to use one over the other?
5. Explain the concept of mutability and its impact on list operations, especially when using lists as default arguments in functions.
6. How would you efficiently find the second-largest element in a list without sorting it?
7. Write a program to rotate a list to the right by `k` steps.
8. How do you create a two-dimensional list (a list of lists)? Give an example of how to access and modify an element in a nested list.
9. Explain the difference in behavior when using `is` vs. `==` to compare two lists.
10. What is a generator expression, and how does it differ from a list comprehension in terms of memory usage?